O Algoritmo SHA-256

Segurança computacional
Prof. Paulo Henrique Lopes Silva

CONCEITOS FUNDAMENTAIS

O que é a Função de Hash SHA-256?

Unidirecional

Fácil calcular o hash; matematicamente impossível reverter para o texto original.

Tamanho Fixo

Produz sempre um resultado de exatamente 256 bits (64 caracteres hexadecimais).

Efeito Avalanche

Qualquer alteração mínima na entrada resulta em um hash totalmente diferente.

VISÃO GLOBAL

Fluxograma Geral do Processamento



PASSO 1 DE 7

Passo 1: Conversão para Código Binário (ASCII)

A mensagem de texto é convertida em seus equivalentes numéricos na tabela ASCII e representada em bits:

Letra 'a': ASCII 97 → `01100001`

Letra 'b': ASCII 98 → `01100010`

Letra 'c': ASCII 99 → `01100011`

```
// Mensagem Inicial Concentrada
01100001 01100010 01100011

Tamanho Original (L) = 24 bits (3 bytes)
```

PASSO 2 DE 7

Passo 2: Preenchimento com Bit '1' e Zeros

Bit de Separação (Bit '1')

Adiciona-se sempre um único bit **1** logo após o fim da mensagem.

Série de Zeros (Bit '0')

Adicionam-se bits **0** até que o comprimento chegue a exatamente **448 bits**.

Motivo do Preenchimento

Reservam-se os últimos 64 bits do bloco de 512 para registrar o tamanho original da mensagem.

ESTRUTURA PARCIAL (448 BITS)

```
[24 bits] Mensagem: 011000010110001001100011

[1 bit] Divisor: 1

[423 bits] Preenchimento: 000000...0000
```

PASSO 3 DE 7

Passo 3: Anexo da Extensão da Mensagem Original

Nos últimos 64 bits, codifica-se o tamanho original da mensagem em binário (64-bit Big-Endian):

Tamanho original: 24 bits.

Valor em Hexadecimal: `0x0000000000000018`

Resultado do Bloco: Exatamente **512 bits** prontos para o cálculo.

```
// Bloco Completo Processável (512 bits)
Bits 0-23: Texto "abc"
Bits 24-447: Bit '1' + Zeros
Bits 448-511: Tamanho Original (24 em binário)
```

PASSO 4 DE 7

Passo 4: Registradores Iniciais (IV) e Constantes K

8 Registradores Iniciais (H0 a H7)

Primeiros 32 bits das partes fracionárias das raízes quadradas dos primeiros 8 primos (2, 3, 5, 7, 11...).

```
H0 = 6a09e667 H1 = bb67ae85
H2 = 3c6ef372 H3 = a54ff53a
H4 = 510e527f H5 = 9b05688c
H6 = 1f83d9ab H7 = 5be0cd19
```

64 Constantes de Rodada (K0 a K63)

Primeiros 32 bits das partes fracionárias das raízes cúbicas dos primeiros 64 números primos.

```
K0 = 428a2f98 K1 = 71374491
K2 = b5c0fbcf K3 = e9b5dba5
...
K63 = c67178f2
```

PASSO 4 DE 7

Valores Completos das 64 Constantes K

Array Completo de Constantes de Rodada (K0 a K63)

Primeiros 32 bits das partes fracionárias das raízes cúbicas dos primeiros 64 números primos representados em hexadecimal.

```
K00: 428a2f98    K16: e49b69c1    K32: 27b70a85    K48: 19a4c116
K01: 71374491    K17: efbe4786    K33: 2e1b2138    K49: 1e376c08
K02: b5c0fbcf    K18: 0fc19dc6    K34: 4d2c6dfc    K50: 2748774c
K03: e9b5dba5    K19: 240ca1cc    K35: 53380d13    K51: 34b0bcb5
K04: 3956c25b    K20: 2de92c6f    K36: 650a7354    K52: 391c0cb3
K05: 59f111f1    K21: 4a7484aa    K37: 766a0abb    K53: 4ed8aa4a
K06: 923f82a4    K22: 5cb0a9dc    K38: 81c2c92e    K54: 5b9cca4f
K07: ab1c5ed5    K23: 76f988da    K39: 92722c85    K55: 682e6ff3
K08: d807aa98    K24: 983e5152    K40: a2bfe8a1    K56: 748f82ee
K09: 12835b01    K25: a831c66d    K41: a81a664b    K57: 78a5636f
K10: 243185be    K26: b00327c8    K42: c24b8b70    K58: 84c87814
K11: 550c7dc3    K27: bf597fc7    K43: c76c51a3    K59: 8cc70208
K12: 72be5d74    K28: c6e00bf3    K44: d192e819    K60: 90befffa
K13: 80deb1fe    K29: d5a79147    K45: d6990624    K61: a4506ceb
K14: 9bdc06a7    K30: 06ca6351    K46: f40e3585    K62: bef9a3f7
K15: c19bf174    K31: 14292967    K47: 106aa070    K63: c67178f2
```

PASSO 5 DE 7

Passo 5: Escalonamento da Mensagem (Message Schedule)

Palavras Diretas (W0 a W15)

O bloco de 512 bits de entrada é fatiado diretamente em 16 palavras sequenciais de 32 bits cada, sem alterações.

```
W[0] = Bloco[0..31]
W[1] = Bloco[32..63]
...
W[15] = Bloco[480..511]
```

Palavras Expandidas (W16 a W63)

As próximas 48 palavras são geradas por meio de uma relação de recorrência matemática com funções de rotação lógica (σ0 e σ1).

```
// Fórmula de Expansão (16 ≤ i ≤ 63)
W[i] = σ1(W[i-2]) + W[i-7] +
σ0(W[i-15]) + W[i-16]
```

PASSO 6 DE 7

Passo 6: As 64 Rodadas de Compressão

Variáveis de trabalho (**a, b, c, d, e, f, g, h**) iniciam como H0..H7 e passam por 64 iterações:

Função Ch(e, f, g)
"Escolha" bitwise baseada no valor de **e**.

Função Maj(a, b, c)
"Maioria" retorna o bit mais frequente entre os 3.

Soma Módulo 2^{32}
Descarta bits excedentes (overflow) para limitar o resultado estritamente a 32 bits.

ATUALIZAÇÃO POR RODADA (t)

```
T1 = h + Σ1(e) + Ch(e,f,g) + K[t] + W[t]
T2 = Σ0(a) + Maj(a,b,c)
```

```
// Atribuição dos novos valores
h = g; g = f; f = e;
e = d + T1;
d = c; c = b; b = a;
a = T1 + T2;
```

PASSO 7 DE 7

Passo 7: Acúmulo e Digest Final de 256 bits

Acúmulo de Variáveis

Após a 64ª rodada, os resultados intermediários (a..h) são somados aos registradores iniciais (H0..H7) usando adição módulo 2^{32}:

```
// Soma dos registradores (A..H)
H0_final = H0 + a
H1_final = H1 + b
...
H7_final = H7 + h
```

Digest Final de 256 bits

Os resultados finais são concatenados na ordem exata de H0 a H7 para produzir o hash final de 32 bytes em representação hexadecimal:

HASH HEXADECIMAL RESULTANTE

```
ba7816bf8f01cfea414140de5dae2223
b00361a396177a9cb410ff61f20015ad
```

Concatenação exata de H0_final + H1_final + ... + H7_final.

DEMONSTRAÇÃO

Demonstração do Efeito Avalanche

ENTRADA 1

"abc"

Uma entrada simples de texto de 3 caracteres produz o hash de 256 bits abaixo:

HASH HEXADECIMAL RESULTANTE

```
ba7816bf8f01cfea414140de5dae2223
b00361a396177a9cb410ff61f20015ad
```

ENTRADA 2 (+1 PONTO)

"abc."

A inserção de apenas um ponto final (.) gera um hash completamente diferente e imprevisível:

HASH HEXADECIMAL RESULTANTE

```
ba80aee0193072314e1fe2a0e4171a06
2e08611311018f8074212d2b6389f2a0
```

LIMITAÇÕES E CAPACIDADES

Além do "abc": Suporte a Mensagens Longas e Limites Teóricos

Mensagens Maiores que "abc"

O algoritmo SHA-256 não se limita a entradas pequenas de texto. Ele foi projetado para processar sequências massivas de dados de entrada dividindo-as em blocos funcionais.

- O texto de entrada é fatiado e estruturado em blocos de 512 bits.
- Se a entrada for menor, é feito o preenchimento (padding) automático de bits.
- Arquivos inteiros, imagens ou bancos de dados podem ser mapeados para um único hash estável.

COMO O ENTRADA É PROCESSADO (BLOCO DE 512 BITS)

```
// Fatiamento em palavras de 32 bits
W[0] = Bloco[0..31]
W[1] = Bloco[32..63]
...
W[15] = Bloco[480..511]
```

Tamanho Máximo de Mensagem por Versão

Cada variante da família SHA possui limites estruturais rígidos para o tamanho máximo da mensagem de entrada, definidos pelo registrador de tamanho anexado na preparação:

SHA-224
$< 2^{64}$ bits
~2,1 Exabytes

SHA-256
$< 2^{64}$ bits
~2,1 Exabytes

SHA-384
$< 2^{128}$ bits
~$4{,}2 \times 10^{38}$ bytes

SHA-512
$< 2^{128}$ bits
~$4{,}2 \times 10^{38}$ bytes

REFERÊNCIAS

Referências Bibliográficas

1. **NAKAMOTO, Satoshi.** Bitcoin: A Peer-to-Peer Electronic Cash System. [S. l.], 2008. Disponível em: <https://bitcoin.org/bitcoin.pdf>. Acesso em: 22 set. 2026.
2. **NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST).** FIPS PUB 180-4: Secure Hash Standard (SHS). Gaithersburg: NIST, 2015. Disponível em: <https://doi.org/10.6028/NIST.FIPS.180-4>. Acesso em: 22 set. 2026.
3. **RESUHR, Eric; SCHNEIER, Bruce.** Applied Cryptography: Protocols, Algorithms, and Source Code in C. 2. ed. New York: John Wiley & Sons, 1996.
4. **RESUHR, E. K. et al.** RFC 6234: US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF). Internet Engineering Task Force (IETF), 2011. Disponível em: <https://tools.ietf.org/html/rfc6234>. Acesso em: 22 set. 2026.